Network Security Sets via Algorithmic Game Theory

Final Report (Tasks 0–4)

Yaekob Beyene Yowhanns
Matricola: 269860
DeMaCS, UniCal

May 27, 2026

Contents

1 Problem definition and notation

Let $G = (V, E)$ be an undirected graph. For $v \in V$, let $N(v)$ denote the neighbor set. A subset $S \subseteq V$ is a *Network Security Set (NSS)* if

$$\forall v \in V : \quad v \in S \quad \vee \quad N(v) \subseteq S. \tag{1}$$

An NSS S is *minimal* (inclusion-wise) if removing any node breaks the NSS property:

$$S \text{ is minimal} \iff S \text{ satisfies (1) and } \forall i \in S : \; S \setminus \{i\} \text{ is not an NSS.} \tag{2}$$

Throughout the report, *minimal* means inclusion-wise minimal, not necessarily minimum-cardinality.

2 Task 0: Setup, graph generation, and validation

2.1 Graph generation

The implementation generates reproducible graphs using a fixed random seed. Three graph families are supported:

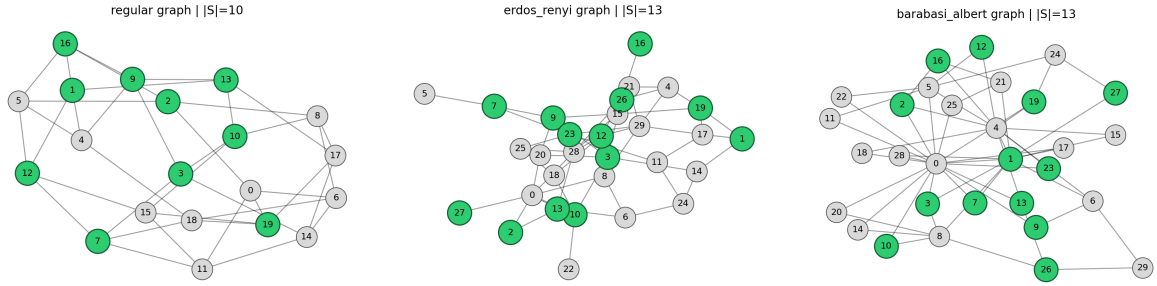
- **k-regular graphs** with parameters (n, k), feasible when nk is even;
- **Erdős–Rényi graphs** with parameters (n, p), using independent edge probability p;
- **Barabási–Albert graphs** with parameters (n, m), using preferential attachment with m edges per new node.

For large-scale ER experiments, a degree-controlled setting is used:

$$p = \frac{4}{n-1} \quad \Rightarrow \quad \mathbb{E}[\deg(v)] \approx 4, \tag{3}$$

So ER density is comparable to $k = 4$ regular graphs.

After generation, nodes are relabeled to integers $\{0, \ldots, n-1\}$. The same graph-generation procedure is used by both the Streamlit interface and the CLI runners. Fixed seeds make the experiments reproducible and allow fair comparisons between methods on the same graph instances.



(a) Regular graph sample. (b) Erdős–Rényi sample. (c) Barabási–Albert sample.

Figure 1: Graph samples generated by the implementation. Highlighted nodes show a sample selected set for visualization.

2.2 Security set validators

Two validators are implemented and used throughout Tasks 1–4:

- `is_security_set(G,S)` checks (1).
- `is_minimal_security_set(G,S)` checks (2).

2.3 Experimental Infrastructure: Streamlit and CLI Runners

The implementation provides two execution modes. The Streamlit interface is used for interactive visualization, inspection, and small-to-medium diagnostic examples. It is useful for displaying graph states, algorithm histories, and build–prune steps. The command-line interface is used for reproducible experiments, larger graph sizes, and CSV export, where storing full histories would be inefficient.

Three CLI runners are used:

- `run_task1_cli.py`: runs the Task 1 methods BRD, Regret Matching, and Fictitious Play.
- `run_task2_cli.py`: runs the Task 2 coalitional approach, including v_1, v_2, v_3, Shapley approximation, build phase, and pruning phase.
- `run_task12_compare_cli.py`: compares Task 1 and Task 2 on the same graph instance using common final-output metrics.

The UI and CLI use the same underlying algorithms and validators. Therefore, when the same graph, seed, and parameters are used, the expected logical outputs are the same: final set size, validity, minimality, and convergence status. The main difference is runtime, because the UI keeps more diagnostic information for visualization, while the CLI uses compact logging. For this reason, runtimes are interpreted only within the execution mode in which they were measured.

In Task 1 CLI experiments, RM and FP use `store_history=False` by default, so full action profiles are not stored at every iteration. In Task 2 CLI experiments, intermediate minimality logging is disabled by default for efficiency, while final security and final inclusion-wise minimality are always checked. The comparison CLI is used only when Task 1 and Task 2 need to be evaluated on the same graph using shared final metrics such as set size, validity, minimality, coverage, and runtime.

3 Task 1: Strategic game and learning dynamics

3.1 Strategic game

Players are nodes $i \in V$. Each chooses $a_i \in \{0, 1\}$: $a_i = 1$ means i secures itself (so $i \in S$), and $a_i = 0$ means free-ride. Define $S(a) = \{i \in V : a_i = 1\}$.

If $a_i = 0$, player i is *safe* iff all neighbors secure:

$$\text{safe}(i, a) \iff \forall j \in N(i) : a_j = 1.$$

Utilities are:

$$u_i(a) = \begin{cases} -c_i, & a_i = 1, \\ 0, & a_i = 0 \text{ and } \text{safe}(i, a), \\ -P, & a_i = 0 \text{ and } \neg\text{safe}(i, a), \end{cases} \qquad (P \gg c_i > 0), \tag{4}$$

with tie-breaking in favor of $a_i = 0$. This tie-breaking encourages a player to avoid paying the security cost when it is already protected, helping to remove redundant secured nodes.

3.2 Why PNE induce minimal NSS

At a Pure Nash Equilibrium (PNE) $a^\star$, no player can improve by deviating alone. If $v \notin S(a^\star)$ has a neighbor $u \notin S(a^\star)$, then v is unsafe and receives $-P$; deviating to 1 yields $-c_v > -P$, contradiction. Hence $S(a^\star)$ satisfies (1). Moreover, if some $i \in S(a^\star)$ were removable (switch to 0 and remain safe), i would improve from $-c_i$ to 0, contradicting equilibrium. Thus, under $P > c_i$ for all players, every PNE induces an inclusion-wise minimal NSS.

3.3 Learning dynamics (BRD, RM, FP)

Three iterative dynamics are compared for reaching a pure equilibrium. All three methods are evaluated by the same external validation step: after each iteration/pass, the resulting pure action profile is checked for PNE and the induced set $S(a)$ is checked using the NSS validators.

Best Response Dynamics (BRD). BRD updates players sequentially. At its turn, a node observes the current action profile and switches to its best response. In this game, a node secures itself if not securing would leave it unsafe; otherwise it free-rides. This method is deterministic apart from the randomized visiting order and is the closest direct procedure for constructing a minimal NSS.

Regret Matching (RM). RM maintains cumulative regrets for both actions 0 and 1. At each iteration, a player compares the utility of the action it actually played with the counterfactual utility it would have obtained by playing the other action. The next action is sampled proportionally to positive cumulative regret. If both positive regrets are zero, the player randomizes uniformly. RM is therefore a stochastic learning dynamic; it does not call the best-response routine.

Fictitious Play (FP). FP uses empirical beliefs instead of the current profile. For each node j, the belief p_j is the empirical frequency with which j has played action 1. A node i estimates the probability of being safe without securing as

$$\Pr(\text{safe}_i) = \prod_{j \in N(i)} p_j,$$

using an independence approximation over neighbor beliefs. It then compares

$$EU_i(1) = -c_i, \qquad EU_i(0) = -P\big(1 - \Pr(\text{safe}_i)\big).$$

In the reported experiments, FP is run in sequential mode, so beliefs are updated immediately after each player's action within a pass.

3.3.1 Algorithmic implementation of BRD, RM, and FP

This subsection gives the pseudocode used for the three Task 1 dynamics. Each method starts from an initial pure action profile $a^0 \in \{0,1\}^{|V|}$ and returns a final profile a. The induced security set is $S(a) = \{i \in V : a_i = 1\}$. After each iteration or full pass, the current profile is checked for PNE; when a PNE is reached, $S(a)$ is validated using `is_security_set` and `is_minimal_security_set`. This PNE check is used as an external stopping and validation criterion for all three methods.

Best Response Dynamics. BRD is the most direct dynamics. Players are visited sequentially, optionally in a shuffled order. Each visited node evaluates the two possible actions against the current profile and immediately switches to the payoff-maximizing action. Therefore, later nodes in the same pass observe earlier updates.

Algorithm 1 Best Response Dynamics (BRD)

Require: Graph $G = (V, E)$, initial profile a^0, costs c_i, penalty P, maximum passes T
Ensure: Final action profile a and induced set $S(a)$

$a \leftarrow a^0$
if a is a PNE **then**
    return a
end if
for $t = 1, \ldots, T$ **do**
    Choose an order of the nodes, optionally shuffled
    for each node i in the chosen order **do**
        Compute $u_i(0, a_{-i})$ and $u_i(1, a_{-i})$ using (4)
        if $u_i(1, a_{-i}) > u_i(0, a_{-i})$ **then**
            $a_i \leftarrow 1$
        else
            $a_i \leftarrow 0$ ▷ tie-breaking favors free-riding
        end if
    end for
    if a is a PNE **then**
        return a
    end if
end for
return a

Interpretation. BRD directly uses the current action profile. Unsafe nodes prefer securing because $-c_i > -P$, while protected nodes prefer free-riding because $0 > -c_i$. Thus, BRD locally removes unsafe configurations and redundant secured nodes.

Regret Matching. RM is a stochastic learning dynamic. The reported experiments use the synchronous version: regrets are first updated for all players using the current profile, and then a new profile is sampled. RM does not call the best-response routine; action probabilities are obtained only from positive cumulative regret.

Algorithm 2 Pure Regret Matching (RM)

Require: Graph $G = (V, E)$, initial profile a^0, costs c_i, penalty P, iterations T
Ensure: Final action profile a and induced set $S(a)$

$a \leftarrow a^0$
for each node $i \in V$ **do**
    $R_i(0) \leftarrow 0$, $R_i(1) \leftarrow 0$
end for
if a is a PNE **then**
    return a
end if
for $t = 1, \ldots, T$ **do**
    for each node $i \in V$ **do**
        Compute counterfactual utilities $u_i(0, a_{-i})$ and $u_i(1, a_{-i})$
        Let $u_i(a_i, a_{-i})$ be the utility of the action actually played
        for each action $b \in \{0, 1\}$ **do**
            $R_i(b) \leftarrow R_i(b) + u_i(b, a_{-i}) - u_i(a_i, a_{-i})$
        end for
    end for
    for each node $i \in V$ **do**
        $R_i^+(0) \leftarrow \max\{R_i(0), 0\}$, $R_i^+(1) \leftarrow \max\{R_i(1), 0\}$
        $Z_i \leftarrow R_i^+(0) + R_i^+(1)$
        if $Z_i = 0$ **then**
            Sample a_i uniformly from $\{0, 1\}$
        else
            Sample $a_i = b$ with probability $R_i^+(b)/Z_i$
        end if
    end for
    if a is a PNE **then**
        return a
    end if
end for
return a

Interpretation. If a node previously played 0 while unsafe, regret for action 1 increases strongly. If a node secured while it was already protected, regret for action 0 increases. Since the unsafe penalty P is much larger than the security cost c_i, RM may require many iterations before the sampled pure profile reaches a PNE.

Fictitious Play. FP is a belief-based learning dynamic. The reported experiments use sequential FP: after a node acts, its empirical frequency is updated immediately. A node then decides using beliefs about how often its neighbors have secured in the past, rather than only using the current action profile.

Algorithm 3 Sequential Fictitious Play (FP)

Require: Graph $G = (V, E)$, initial profile a^0, costs c_i, penalty P, iterations T
Ensure: Final action profile a and induced set $S(a)$

$a \leftarrow a^0$
for each node $j \in V$ **do**
    $n_j \leftarrow 1$ ▷ number of observations of node j
    $m_j \leftarrow a_j$ ▷ number of times node j has played action 1
    $p_j \leftarrow m_j / n_j$
end for
if a is a PNE **then**
    return a
end if
for $t = 1, \ldots, T$ **do**
    Choose an order of the nodes, optionally shuffled
    for each node i in the chosen order **do**
        $q_i \leftarrow \prod_{j \in N(i)} p_j$ ▷ estimated probability of being safe without securing
        $EU_i(1) \leftarrow -c_i, \quad EU_i(0) \leftarrow -P(1 - q_i)$
        if $EU_i(1) > EU_i(0)$ **then**
            $a_i \leftarrow 1$
        else
            $a_i \leftarrow 0$ ▷ tie-breaking favors free-riding
        end if
        $n_i \leftarrow n_i + 1, \quad m_i \leftarrow m_i + a_i, \quad p_i \leftarrow m_i / n_i$
    end for
    if a is a PNE **then**
        return a
    end if
end for
return a

Interpretation. FP differs from BRD because it responds to empirical beliefs. A node secures when the historical behavior of its neighbors makes free-riding too risky. This explains why sequential FP can be effective, but also why it can be iteration-cap sensitive on larger sparse graphs.

3.4 Experimental setup for Task 1

The reported UI tables use seed 42 and show representative runs with zero initialization and random initialization with $p_0 = 0.30$, as indicated in each row. The default iteration cap is 1000 unless an extended run is explicitly discussed. For each run, the reported metrics are: whether a PNE was reached, the number of iterations used, the final set size $|S|$, and runtime. When `reached_pne=True`, the output also satisfies `is_security_set=True` and `is_minimal_security_set=True`.

The tables in Sections 3.4.1–3.4.3 report Streamlit UI runs, which were used for detailed inspection and diagnostics. Additional CLI runs were used for scalable validation, CSV export, and the Task 1–Task 2 comparison. Since the UI and CLI use different logging levels, runtimes should be compared only within the same execution mode.

3.4.1 Task 1 results — Regular graphs (k=4)

Table 1: Task 1 results for Regular graphs (k=4), seed 42; init in zeros, random (p0=0.30); max iters 1000; FP sequential.

n	init	BRD			RM			FP		
		PNE	it	$\|S\|$ (time s)	PNE	it	$\|S\|$ (time s)	PNE	it	$\|S\|$ (time s)
1000	zeros	Y	2	673 (0.0488)	Y	354	669 (10.6201)	Y	399	661 (3.0911)
1000	random	Y	2	664 (0.0674)	Y	632	666 (20.1144)	Y	399	656 (2.4755)
5000	zeros	Y	2	3333 (2.2056)	Y	406	3322 (276.3064)	Y	399	3300 (15.9564)
5000	random	Y	2	3336 (1.0270)	Y	990	3313 (486.3345)	Y	399	3293 (12.6159)
10000	zeros	Y	2	6680 (4.3031)	Y	408	6678 (793.6295)	Y	399	6600 (29.3249)
10000	random	Y	2	6702 (4.3572)	N	1000	6646 (1894.8605)	Y	399	6564 (27.0625)

Regular-graph observations. On regular graphs, BRD consistently converges very quickly, reaching a PNE in about 2 iterations in the reported runs. FP also reaches a PNE in all shown regular-graph cases and shows stable behavior under the selected sequential empirical-belief configuration. RM is correct and can also reach a PNE on large regular graphs, but it is computationally heavier, especially for large n and random initialization. In the hardest regular case tested ($n = 10000$, $k = 4$, random initialization with $p = 0.30$), RM did not reach a PNE within the initial cap of 1000 iterations. After increasing the cap to 5000, it reached a PNE at iteration 1236, producing a valid and minimal security set with $|S| = 6642$. Therefore, the earlier non-convergence at $T = 1000$ was due to an insufficient iteration limit, not a failure of RM.

3.4.2 Task 1 results — ER graphs (p = 4/(n-1), expected degree approx 4)

Table 2: Task 1 results for ER graphs (p = 4/(n-1), expected degree approx 4), seed 42; init in zeros, random (p0=0.30); max iters 1000; FP sequential.

n	init	BRD			RM			FP		
		PNE	it	$\|S\|$ (time s)	PNE	it	$\|S\|$ (time s)	PNE	it	$\|S\|$ (time s)
1000	zeros	Y	2	586 (0.0425)	Y	360	596 (7.8892)	Y	697	548 (3.0720)
1000	random	Y	2	603 (0.0325)	Y	679	595 (12.7700)	Y	598	555 (2.3491)
5000	zeros	Y	2	3000 (0.8116)	Y	401	2962 (144.1146)	N	1000	2734 (47.0758)
5000	random	Y	2	2969 (0.8356)	Y	840	2871 (374.8276)	Y	598	2803 (20.7104)
10000	zeros	Y	2	5981 (3.5183)	Y	414	6025 (579.8975)	Y	896	5502 (47.6324)

ER-graph observations. BRD always converges very quickly, typically in two iterations, and returns a valid minimal NSS when it reaches PNE. FP is iteration-cap sensitive on large sparse ER graphs: for $n = 5000$ with `init=zeros`, FP does not reach a PNE within the 1000-iteration budget reported in the table, but when the iteration cap is increased, it reaches a PNE at iteration 1095, and the resulting set is minimal. RM typically reaches a PNE, but in diagnostic UI runs its runtime grows sharply with n.

3.4.3 Task 1 results — BA graphs (m=2)

Table 3: Task 1 results for BA graphs (m=2), seed 42; init in zeros, random (p0=0.30); max iters 1000; FP sequential.

n	init	BRD			RM			FP		
		PNE	it	$\|S\|$ (time s)	PNE	it	$\|S\|$ (time s)	PNE	it	$\|S\|$ (time s)
1000	zeros	Y	2	504 (0.0772)	Y	393	492 (11.9452)	Y	697	436 (3.9939)
1000	random	Y	2	503 (0.0574)	Y	869	478 (23.6367)	Y	498	444 (2.7321)
5000	zeros	Y	2	2490 (1.1295)	Y	399	2543 (207.4460)	N	1000	2237 (31.5903)
5000	random	Y	2	2531 (1.0875)	Y	774	2353 (373.9147)	Y	697	2273 (23.9501)
10000	zeros	Y	2	4912 (4.2862)	Y	396	5071 (856.6494)	Y	896	4357 (102.0284)

BA-graph observations. BRD converges rapidly, typically in one or two iterations, and returns a valid minimal NSS when it reaches PNE. FP is also cap-sensitive on BA graphs: for $n = 5000$, $m = 2$, with `init=zeros`, FP does not reach a PNE within 1000 iterations, but when the iteration cap is increased, it reaches a PNE at iteration 1195, and the resulting set is minimal. RM reaches a PNE in the tested BA instances, but it is computationally heavier in diagnostic UI runs, especially for larger graphs.

3.5 Task 1 takeaway

Across the tested graph families, BRD is the most efficient and robust method. It reaches PNE very quickly and consistently produces valid inclusion-wise minimal security sets. FP often produces smaller final sets than BRD, especially on ER and BA graphs, but it can require a larger iteration budget on some sparse instances.

RM is a valid stochastic learning dynamic, but it is more sensitive to runtime and iteration limits. In diagnostic UI runs, RM is the most computationally expensive method because more intermediate information is maintained. In compact CLI mode, RM is faster because full histories are not stored and counterfactual utilities are computed locally, but it can still require many stochastic iterations before the sampled pure profile reaches a PNE.

4 Task 2: Coalitional Game and Shapley-Induced NSS

4.1 Coalitional model

Task 2 studies the Network Security Set problem using a coalitional-game perspective. The game is defined as (V, v), where the players are the nodes of the graph and the characteristic function v assigns a value to every coalition $C \subseteq V$.

In this setting, a coalition C represents the set of nodes that install security devices. Therefore, C is interpreted as a candidate set of secured nodes.

Satisfaction and violations. Given a coalition $C \subseteq V$, a node $x \in V$ is **satisfied** if it is secured itself or if all of its neighbors are secured:

$$x \text{ is satisfied under } C \iff (x \in C) \ \vee \ (N(x) \subseteq C).$$

A node x is a **violation** if it is not secured and is not fully protected by secured neighbors:

$$x \text{ is a violation under } C \iff (x \notin C) \ \wedge \ (N(x) \not\subseteq C).$$

Thus, a coalition C is a Network Security Set exactly when it has no violations:

$$C \text{ is an NSS} \iff \text{violations}(C) = 0.$$

4.2 Coalition features

To evaluate coalitions in a normalized way, three features are computed:

$$f_{\text{sat}}(C) = \frac{\#\{\text{satisfied nodes under } C\}}{|V|},$$

$$f_{\text{edge}}(C) = \frac{\#\{\text{secured edges under } C\}}{|E|},$$

$$f_{\text{size}}(C) = \frac{|C|}{|V|}.$$

An edge $(u, v) \in E$ is counted as **secured** if at least one endpoint is secured:

$$(u, v) \text{ is secured under } C \iff u \in C \ \vee \ v \in C.$$

The feature f_{sat} is the one directly connected to the formal NSS condition. In particular, $f_{\text{sat}}(C) = 1$ exactly when all nodes are satisfied, i.e., when C is a valid NSS. The feature f_{edge} is an auxiliary coverage metric: it measures how much of the network is incident to at least one secured endpoint, but formal NSS feasibility is still determined by node satisfaction and violations. Finally, f_{size} measures resource usage, namely the fraction of nodes selected as secured nodes.

4.3 Characteristic functions

A characteristic function $v(C)$ defines how valuable a coalition is. In this implementation, all characteristic functions are designed as normalized, non-negative security scores. Higher values correspond to coalitions with better node satisfaction, edge coverage, or size-efficient security.

v_1: node-satisfaction score. The first characteristic function is

$$v_1(C) = f_{\text{sat}}(C).$$

This is the simplest function and is directly aligned with the formal NSS condition. If $v_1(C) = 1$, then every node is satisfied and C is a valid NSS. Smaller values indicate that some nodes remain uncovered.

v_2: combined node-and-edge security score. The second characteristic function is

$$v_2(C) = \alpha f_{\text{sat}}(C) + (1 - \alpha) f_{\text{edge}}(C), \qquad \alpha \in [0, 1].$$

The first term rewards formal node satisfaction, while the second term rewards edge-level coverage. The parameter α controls the trade-off. Larger values of α give more importance to node satisfaction, while smaller values give more importance to secured edges.

v_3: size-efficient security score. The third characteristic function is

$$v_3(C) = v_2(C) \cdot \frac{1}{1 + \gamma f_{\text{size}}(C)}, \qquad \gamma \geq 0.$$

The multiplicative factor

$$\frac{1}{1 + \gamma |C|/|V|}$$

down-weights large coalitions and therefore rewards more compact coalitions. This avoids subtracting a penalty, so the coalition value remains non-negative. However, v_3 does not by itself guarantee that the final security set is minimal. Final inclusion-wise minimality is enforced later by the pruning phase.

4.4 Discussion of alternative characteristic functions

Beyond the implemented functions v_1, v_2, and v_3, other characteristic functions can be designed depending on the security assumptions and priorities of the network. The following examples are not used in the experiments, but they show how the coalitional model can be adapted to more specific security scenarios.

Weighted node-satisfaction function. The implemented function v_1 treats all satisfied nodes equally. In some networks, however, nodes may have different predefined importance before coalition evaluation. For example, servers, routers, gateways, high-degree nodes, or high-risk nodes may be more critical than ordinary nodes. Let $w_i > 0$ denote an exogenous importance weight of node i, assigned from network information such as degree, role, risk, or operational criticality. A weighted satisfaction function can be defined as

$$v_{\text{weighted}}(C) = \frac{\sum_{i \in V} w_i \cdot \mathbf{1}[i \text{ is satisfied under } C]}{\sum_{i \in V} w_i}.$$

This function gives higher value to coalitions that satisfy critical nodes, not only coalitions that satisfy many nodes. The importance weights are not produced by the coalition; they are predefined properties of the network used when evaluating the coalition.

Risk-aware violation function. Another possible design is to penalize uncovered nodes according to their risk. In the implemented functions, all violations are treated uniformly. In practice, leaving a high-risk node uncovered may be more serious than leaving a low-risk node uncovered. Let $r_i \geq 0$ denote an exogenous risk score of node i. A risk-aware characteristic function can be written as

$$v_{\text{risk}}(C) = f_{\text{sat}}(C) \cdot \left(1 - \frac{\sum_{i \in V} r_i \cdot \mathbf{1}[i \text{ is a violation under } C]}{\sum_{i \in V} r_i}\right).$$

This function decreases the value of a coalition more strongly when the uncovered nodes are high-risk. A nonlinear version can also be defined as

$$v_{\text{risk-nonlinear}}(C) = f_{\text{sat}}(C) \cdot \left(1 - \left(\frac{\sum_{i \in V} r_i \cdot \mathbf{1}[i \text{ is a violation under } C]}{\sum_{i \in V} r_i}\right)^2\right).$$

The nonlinear form gives a stronger penalty when the coalition leaves many high-risk nodes uncovered. This may be useful in security settings where a large unsafe region is significantly worse than a small number of isolated violations.

Reliability-based characteristic function. The implemented functions assume that a secured node always provides protection. A more realistic model may consider that security devices can fail or may have different levels of reliability. Let $\rho_j \in [0, 1]$ denote the reliability of the security device installed at node j. A reliability-based characteristic function can be defined as

$$v_{\text{rel}}(C) = \frac{1}{|V|} \sum_{i \in V} \Pr(i \text{ is protected by } C).$$

One possible protection model is

$$\Pr(i \text{ is protected by } C) = \begin{cases} \rho_i, & i \in C, \\ \prod_{j \in N(i)} \rho_j, & i \notin C \text{ and } N(i) \subseteq C, \\ 0, & \text{otherwise.} \end{cases}$$

This function is useful when protection is uncertain or when different secured nodes provide different security quality. For example, a coalition containing high-reliability secured nodes would receive a higher value than a coalition that relies on low-reliability protection.

Combined weighted-risk-size function. A more complete characteristic function could combine node importance, risk awareness, and size efficiency:

$$v_{\text{combined}}(C) = \left(\frac{\sum_{i\in V} w_i \cdot \mathbf{1}[i \text{ is satisfied under } C]}{\sum_{i\in V} w_i}\right)\cdot\left(1 - \frac{\sum_{i\in V} r_i \cdot \mathbf{1}[i \text{ is a violation under } C]}{\sum_{i\in V} r_i}\right)\cdot\frac{1}{1+\gamma|C|/|V|}$$

This function rewards coalitions that satisfy important nodes, avoid high-risk violations, and remain compact. It combines the ideas of weighted satisfaction, risk-aware security, and size efficiency in one coalition-value function.

These alternative characteristic functions are realistic and implementable, but they introduce additional parameters such as node-importance weights, risk scores, reliability values, and penalty strengths. For this reason, the implemented functions v_1, v_2, and v_3 are kept simple, normalized, non-negative, and easier to compare experimentally.

4.5 Shapley-value approximation

For a characteristic function v, the Shapley value of node i is

$$\phi_i = \sum_{C\subseteq V\setminus\{i\}} \frac{|C|!(|V|-|C|-1)!}{|V|!}\Big(v(C\cup\{i\}) - v(C)\Big).$$

It measures the average marginal contribution of node i over all possible orders in which nodes can join a coalition.

Exact Shapley-value computation is exponential in the number of nodes. Therefore, the implementation uses Monte Carlo approximation. It samples K random permutations of the nodes. For each permutation, nodes are added one by one to a growing coalition, and the marginal contribution of each node is computed. The final Shapley score is the average marginal contribution over the sampled permutations.

4.6 Inducing a minimal NSS from the Shapley ranking

After estimating Shapley values, nodes are sorted in descending Shapley order. This ranking is then used to construct a security set in two phases.

Build phase. Starting from $S = \emptyset$, nodes are added one by one according to the Shapley ranking until the selected set first becomes a valid NSS. The size of this first valid candidate is denoted by

$$|S|_{\text{build}}.$$

This metric evaluates how many high-Shapley nodes were needed before the NSS condition was satisfied.

Prune phase. The build phase can produce a valid but non-minimal NSS. Therefore, a pruning phase removes redundant nodes. A node $i \in S$ is removed if $S \setminus \{i\}$ is still a valid NSS. The number of removed nodes is

$$\text{pruned} = |S|_{\text{build}} - |S|_{\text{final}}.$$

The final set is checked using both:

`is_security_set` and `is_minimal_security_set`.

Thus, Shapley values are used as a principled node-ranking mechanism, while the pruning phase enforces inclusion-wise minimality. The build size measures the quality of the ranking, the final size measures the final solution, and the pruned count measures how much redundancy remained after the build phase.

4.7 Worked example

Consider the graph

$$V = \{1, 2, 3, 4, 5\}$$

with edges

$$E = \{(1, 2), (1, 3), (1, 4), (1, 5), (2, 3)\}.$$

For the coalition $C = \{1\}$, node 1 is directly secured. Nodes 4 and 5 are also satisfied because their only neighbor is node 1. However, nodes 2 and 3 are not satisfied because neither of them is secured and each has an unsecured neighbor. Therefore,

$$f_{\text{sat}}(C) = \frac{3}{5}.$$

The four edges incident to node 1 are secured, while edge $(2, 3)$ is not:

$$f_{\text{edge}}(C) = \frac{4}{5}, \qquad f_{\text{size}}(C) = \frac{1}{5}.$$

For the coalition $C = \{1, 2, 3\}$, nodes $1, 2, 3$ are directly secured, while nodes 4 and 5 are protected by node 1. Therefore, this coalition is a valid NSS:

$$f_{\text{sat}}(C) = 1, \qquad f_{\text{edge}}(C) = 1, \qquad f_{\text{size}}(C) = \frac{3}{5}.$$

This example illustrates why size efficiency is useful: two coalitions may both provide strong security coverage, but the smaller coalition is preferable when a similar level of protection is achieved with fewer secured nodes.

4.8 Experimental protocol

The Task 2 experiments evaluate whether the Shapley-induced ranking can be used to construct valid and minimal Network Security Sets. For each graph family, graph size, seed, and characteristic function, the following pipeline is used:

$v(C) \rightarrow$ Monte Carlo Shapley values $\rightarrow$ descending node ranking $\rightarrow$ build until NSS $\rightarrow$ prune redundant nodes

The reported metrics are:

- **Success**: fraction of runs where the final set is a valid NSS.
- **Minimal**: fraction of runs where the final set is inclusion-wise minimal.
- **Build mean**: average size of the first Shapley-induced set that becomes an NSS.
- **Final $|S|$ mean**: average size after pruning.
- **Pruned mean**: average number of redundant nodes removed.
- **Runtime**: average runtime in seconds.

The experiments use $K = 500$ Monte Carlo Shapley samples, $\alpha = 0.7$, $\gamma = 1.0$, and 10 seeds from 42 to 51.

The tables in Sections 4.9–4.11 report Streamlit UI runs over multiple seeds, while Section 4.13 reports a separate CLI comparison on the same graph instances. The algorithms are the same, but CLI mode uses compact logging, so runtime values should be interpreted separately from UI runtimes.

4.9 Task 2 experiments on regular graphs

Regular graphs with degree $k = 4$ were tested for $n = 100$ and $n = 200$.

n	k	v	Success	Minimal	Build mean	Final $\lvert S\rvert$ mean	Pruned mean	Runtime (s)
100	4	v_1	1.00	1.00	90.8	66.0	24.8	9.41
100	4	v_2	1.00	1.00	89.5	66.6	22.9	9.45
100	4	v_3	1.00	1.00	87.9	66.6	21.3	8.98
200	4	v_1	1.00	1.00	183.9	133.0	50.9	34.18
200	4	v_2	1.00	1.00	178.6	133.3	45.3	34.09
200	4	v_3	1.00	1.00	181.5	132.3	49.2	35.75

Table 4: Task 2 Shapley-induced NSS results on regular graphs with $k = 4$.

All three characteristic functions achieved success rate 1.00 and minimality rate 1.00. Therefore, every final set was a valid inclusion-wise minimal NSS after pruning.

The final sizes are very similar across v_1, v_2, and v_3: around 66 nodes for $n = 100$ and around 132–133 nodes for $n = 200$. Thus, on regular graphs with degree 4, the final sets contain about two thirds of the nodes. This is consistent with the Task 1 results on regular graphs.

The build sizes are much larger than the final sizes. For example, when $n = 200$, the average number of removed nodes ranges from 45.3 for v_2 to 50.9 for v_1. This shows that the Shapley ranking reaches a valid NSS, but the built set contains redundant nodes. The pruning phase is therefore essential for obtaining an inclusion-wise minimal NSS.

4.10 Task 2 experiments on Erdős–Rényi graphs

The same construction was tested on Erdős–Rényi graphs. To make them comparable with regular graphs of degree $k = 4$, the edge probability was set to

$$p = \frac{4}{n-1},$$

so the expected degree is approximately 4.

n	p	v	Success	Minimal	Build mean	Final $\lvert S\rvert$ mean	Pruned mean	Runtime (s)
100	0.0404	v_1	1.00	1.00	65.7	55.2	10.5	14.42
100	0.0404	v_2	1.00	1.00	71.2	55.0	16.2	13.63
100	0.0404	v_3	1.00	1.00	71.7	55.3	16.4	14.16
200	0.0201	v_1	1.00	1.00	131.2	107.8	23.4	50.75
200	0.0201	v_2	1.00	1.00	147.2	108.3	38.9	48.52
200	0.0201	v_3	1.00	1.00	145.8	108.3	37.5	49.81

Table 5: Task 2 Shapley-induced NSS results on Erdős–Rényi graphs with $p = 4/(n-1)$.

All three characteristic functions again achieved success rate 1.00 and minimality rate 1.00. The final sizes are stable across v_1, v_2, and v_3: about 55 nodes for $n = 100$ and about 108 nodes for $n = 200$.

These final sizes are smaller than those observed on regular graphs with comparable expected degree. This is reasonable because Erdős–Rényi graphs have degree variability. Some nodes have more useful neighborhoods than others, so selecting such nodes can satisfy more of the network.

The build phase shows clearer differences between the characteristic functions. In these experiments, v_1 reaches a valid NSS with a smaller build size than v_2 and v_3. This suggests that node satisfaction is the most direct objective for reaching the formal NSS condition. The functions v_2 and v_3 also account for edge coverage and size efficiency, which modifies the ranking

and can introduce more redundancy before pruning. For example, when $n = 200$, v_2 first reaches an NSS with average build size 147.2, but pruning reduces the final size to 108.3.

4.11 Task 2 experiments on Barabási–Albert graphs

The Shapley-induced construction was also tested on Barabási–Albert graphs with parameter $m = 2$.

n	m	v	Success	Minimal	Build mean	Final $\lvert S\rvert$ mean	Pruned mean	Runtime (s)
100	2	v_1	1.00	1.00	46.0	44.3	1.7	12.75
100	2	v_2	1.00	1.00	46.8	44.3	2.5	12.43
100	2	v_3	1.00	1.00	47.3	44.3	3.0	12.64
200	2	v_1	1.00	1.00	93.2	88.1	5.1	48.01
200	2	v_2	1.00	1.00	96.2	88.2	8.0	49.16
200	2	v_3	1.00	1.00	97.8	88.0	9.8	49.46

Table 6: Task 2 Shapley-induced NSS results on Barabási–Albert graphs with $m = 2$.

All three characteristic functions achieved success rate 1.00 and minimality rate 1.00. The final set sizes are highly stable across v_1, v_2, and v_3: approximately 44 nodes for $n = 100$ and approximately 88 nodes for $n = 200$.

These values are smaller than those obtained on regular and Erdős–Rényi graphs. This is explained by the hub structure of Barabási–Albert graphs. Hub nodes can contribute to satisfying many other nodes, so they tend to receive high Shapley contribution and are selected early in the ranking.

The build sizes are close to the final sizes, especially for $n = 100$, where only about 1.7–3.0 nodes are removed on average. This indicates that the Shapley ranking is particularly effective on BA graphs. For $n = 200$, pruning remains useful, but the number of removed nodes is still much smaller than in the regular and ER experiments.

4.12 Task 2 takeaway

Across regular, Erdős–Rényi, and Barabási–Albert graphs, all three characteristic functions achieved success rate 1.00 and minimality rate 1.00 after pruning. This confirms that the Shapley-induced build–prune procedure reliably constructs valid inclusion-wise minimal NSS in the tested settings.

The final pruned sizes are usually close across v_1, v_2, and v_3, while the build sizes and pruned counts differ more clearly. This means that the choice of characteristic function mainly affects the Shapley ranking and the amount of redundancy introduced before pruning. The pruning phase is therefore essential: Shapley values guide the construction, while pruning enforces minimality.

Graph structure has a clear effect on the final NSS size. Regular graphs produce the largest final sets, Erdős–Rényi graphs produce smaller sets due to degree variability, and Barabási–Albert graphs produce the smallest sets because hub nodes have high marginal contribution. Thus, Task 2 provides a complementary view to Task 1: instead of modeling selfish best responses, it ranks nodes by their average coalitional contribution to network security.

4.13 Task 1 vs Task 2 CLI Comparison

After evaluating Task 1 and Task 2 separately, an additional CLI experiment was performed to compare the strategic-game methods and the coalitional Shapley-based methods on the same graph instances. The comparison uses common final-output metrics: final set size, security-set validity, inclusion-wise minimality, and runtime. Task 1 uses zero initialization in these comparison runs, while Task 2 does not require an initial action profile.

Table 7 summarizes the comparison for $n = 200$ and $n = 500$, using seed 42, $K = 500$, $\alpha = 0.7$, and $\gamma = 1.0$.

Graph	Method	$n = 200$		$n = 500$	
		$\lvert S\rvert$	Time (s)	$\lvert S\rvert$	Time (s)
Regular	BRD	138	0.00	335	0.00
Regular	RM	133	0.10	333	0.25
Regular	FP-seq	133	0.13	329	0.25
Regular	Shapley(v_1)	129	19.44	335	240.57
Regular	Shapley(v_2)	131	35.84	337	333.45
Regular	Shapley(v_3)	132	16.26	335	338.68
ER	BRD	118	0.00	293	0.01
ER	RM	119	0.13	296	0.55
ER	FP-seq	114	0.34	268	1.14
ER	Shapley(v_1)	114	33.96	262	243.64
ER	Shapley(v_2)	114	52.37	263	220.15
ER	Shapley(v_3)	114	54.23	264	151.75
BA	BRD	104	0.00	265	0.01
BA	RM	94	0.07	262	0.18
BA	FP-seq	83	0.16	231	0.41
BA	Shapley(v_1)	83	28.00	230	211.52
BA	Shapley(v_2)	83	34.63	231	210.84
BA	Shapley(v_3)	83	37.28	231	211.10

Table 7: Task 1 vs Task 2 CLI comparison on the same graph instances. Task 1 uses zero initialization. Task 2 uses $K = 500$, $\alpha = 0.7$, and $\gamma = 1.0$. In all rows, the final set is a valid and inclusion-wise minimal NSS.

The comparison shows a clear trade-off. Task 1 methods are much faster and more scalable. BRD is the fastest method, but it often gives larger minimal sets than FP or the Shapley-based methods. Sequential FP is the strongest Task 1 method in terms of final cardinality, often matching or nearly matching the Shapley-based solutions.

Task 2 sometimes matches or slightly improves the best Task 1 result. For example, on ER with $n = 500$, FP gives $|S| = 268$, while Shapley(v_1) gives $|S| = 262$. On BA with $n = 500$, FP gives $|S| = 231$, while Shapley(v_1) gives $|S| = 230$. However, these improvements come with much higher runtime because Monte Carlo Shapley approximation requires many coalition-value evaluations.

Overall, Task 1 is preferable for large-scale computation, while Task 2 provides a complementary coalitional ranking and interpretability perspective. Shapley-based methods can produce comparable or slightly smaller minimal sets on some graphs, but they are computationally more expensive.

5 Task 3: Planner Assignment to Vendors

5.1 Planner model

Task 3 uses a minimal security set S obtained from the previous tasks. The nodes in S are treated as players who need to acquire security services from vendors. Each player $i \in S$ is assigned a budget

$$b_i \in \{1, \ldots, 100\}.$$

A set of vendors is generated. Each vendor j has a price, a security level, and, in the limited-capacity case, a capacity:

$$p_j \in \{1, \ldots, 100\}, \qquad \ell_j \in \{1, \ldots, 10\}, \qquad c_j \in \mathbb{N}.$$

Here, p_j is the vendor price, ℓ_j is the vendor security level, and c_j is the maximum number of players that vendor j can serve.

Compatibility. A player can be assigned to a vendor only if the vendor price does not exceed the player's budget:

$$p_j \leq b_i.$$

If $p_j > b_i$, the pair (i, j) is infeasible. A dummy option `UNASSIGNED`, with utility 0, is included to represent players who cannot be matched to a feasible or available vendor.

5.2 Normalized non-negative utility

For every compatible player-vendor pair, the utility combines vendor security quality and affordability:

$$u(i, j) = \alpha \left(\frac{\ell_j}{10} \right) + (1 - \alpha) \left(1 - \frac{p_j}{b_i} \right), \qquad \text{defined only when } p_j \leq b_i. \tag{5}$$

The parameter $\alpha \in [0, 1]$ controls the trade-off between security level and affordability:

- larger α gives more importance to vendor security level;
- smaller α gives more importance to affordability.

Since the utility is evaluated only for compatible pairs, $p_j \leq b_i$, both terms in (5) are normalized and non-negative. Therefore,

$$u(i, j) \in [0, 1].$$

The dummy option satisfies

$$u(i, \texttt{UNASSIGNED}) = 0.$$

5.3 Planner objective and assignment cases

The planner seeks an assignment x that maximizes total social welfare:

$$\max_x \sum_{i \in S} u(i, x_i), \tag{6}$$

subject to compatibility and, in the limited-capacity case, vendor capacity constraints.

Case A: infinite supply. In Case A, vendors have unlimited supply. Therefore, each player independently chooses the compatible vendor with maximum utility:

$$x_i \in \arg \max_{j : p_j \leq b_i} u(i, j).$$

If no compatible vendor exists, the player is assigned to `UNASSIGNED`. This independent assignment is welfare-optimal because one player's choice does not affect another player's feasible options.

Case B: limited capacity. In Case B, each vendor j can serve at most c_j players:

$$\#\{i \in S : x_i = j\} \leq c_j.$$

Now player decisions are coupled by vendor capacities, so independent greedy assignment is not sufficient.

The implementation solves this case as a min-cost flow problem. The flow network contains source-to-player edges, player-to-compatible-vendor edges, vendor-to-sink edges with capacities c_j, and a dummy `UNASSIGNED` vendor with capacity $|S|$. For each compatible player-vendor edge (i, j), the cost is set to the negative utility:

$$\text{cost}(i, j) = -u(i, j),$$

scaled to an integer for the solver. Thus, minimizing total flow cost is equivalent to maximizing the welfare objective in (6).

5.4 Experiment 1: Case A versus Case B

As a representative experiment, the security set produced by Task 1 on a regular graph with $n = 200$ and $k = 4$ is used. The selected security set contains

$$|S| = 133$$

players. The planner uses 15 vendors, seed 42, and $\alpha = 0.70$. In the main limited-capacity case, vendor capacities are generated in the range $[1, 5]$.

Case	Players	Welfare	Matched	Unmatched	Avg. utility	Runtime (ms)
A: Infinite supply	133	77.451	129	4	0.582	0.92
B: Limited capacity	133	14.339	36	97	0.108	42.08

Table 8: Task 3 comparison between infinite supply and limited vendor capacity.

In Case A, almost all players are matched because vendors have unlimited supply. Only 4 players remain unmatched because they have no compatible vendor. The total welfare is 77.451, and the average utility over all players is 0.582.

In Case B, vendor capacities strongly restrict the assignment. Only 36 players are matched and 97 are assigned to `UNASSIGNED`. The total welfare decreases from 77.451 to 14.339, corresponding to a welfare drop of approximately 81.5%. This shows that limited vendor capacity is the dominant constraint in this experiment.

5.5 Vendor-load and unmatched-player analysis

In the infinite-supply case, the assignment naturally concentrates on the vendors that provide the highest compatible utility. The main selected vendors are shown in Table 9.

Vendor	Assigned players
V6	98
V4	17
V7	14
`UNASSIGNED`	4

Table 9: Case A vendor load. With infinite supply, players independently choose their best compatible vendor.

This concentration is expected in Case A because vendor supply is unlimited. A large number of players can choose the same high-utility vendor without reducing availability for other players.

In the limited-capacity case, this concentration is no longer possible. Useful vendor capacity becomes saturated. The total real vendor capacity is 41, of which 36 units are used. The remaining unused capacity belongs to two expensive vendors, shown in Table 10.

Vendor	Price	Level	Load	Capacity
V14	99	3	0	3
V0	99	2	0	2

Table 10: Unused vendor capacity in Case B. The unused capacity belongs to high-price, low-level vendors.

Therefore, it is not accurate to say that all vendor capacity is exhausted. A more precise interpretation is that almost all useful and compatible vendor capacity is saturated, while the remaining unused capacity belongs to vendors that are expensive and unattractive under the welfare-maximizing objective.

To understand the unmatched players, each unmatched player in Case B was classified according to compatibility and saturation.

Reason	Number of players
All compatible vendors are saturated	93
No compatible vendor	4

Table 11: Reasons for unmatched players in the limited-capacity case.

The diagnosis shows that the main reason for being unmatched is not lack of compatibility alone. Among the 97 unmatched players, 93 have at least one compatible vendor, but all of their compatible vendors are already saturated. Only 4 players have no compatible vendor at all. Thus, the main bottleneck in Case B is saturation of compatible vendor capacity.

5.6 Experiment 2: Sensitivity to α

The parameter α controls whether the utility function prioritizes vendor security level or affordability. Several values of α were tested while keeping the same planner setting.

α	Welfare A	Welfare B	Drop (%)	Avg. utility B	Matched B	Unmatched B
0.00	106.813	13.792	87.09	0.1037	27	106
0.25	96.235	13.674	85.79	0.1028	32	101
0.50	85.656	13.967	83.69	0.1050	35	98
0.70	77.451	14.339	81.49	0.1078	36	97
0.75	76.516	14.432	81.14	0.1085	36	97
1.00	81.600	14.900	81.74	0.1120	36	97

Table 12: Sensitivity of Task 3 assignment to the utility weight α.

The results show that changing α affects both welfare and the number of matched players. In the limited-capacity case, welfare remains in a narrow range, from 13.674 to 14.900, while the number of matched players increases from 27 at $\alpha = 0.00$ to 36 for $\alpha \geq 0.70$. The best limited-capacity welfare in this sweep is obtained at $\alpha = 1.00$. Thus, in this specific setting, placing more weight on vendor security level improves the limited-capacity outcome slightly, but the improvement is not enough to remove the capacity bottleneck.

Many players remain unmatched for every value of α. Therefore, changing the utility trade-off affects assignment quality, but it does not remove the main bottleneck: limited capacity of compatible vendors.

5.7 Experiment 3: Controlled sensitivity to vendor capacity

A controlled capacity sweep was performed to isolate the effect of vendor capacity. In this experiment, the security set, player budgets, vendor prices, vendor security levels, number of vendors, seed, and α are fixed. Only the capacity assigned to each vendor is changed. Therefore, Case A should remain constant, because the infinite-supply model does not depend on vendor capacity.

Cap./vendor	Total cap.	Welfare A	Welfare B	Drop (%)	Matched B	Unmatched B
1	15	77.451	5.775	92.54	15	118
2	30	77.451	11.230	85.50	29	104
3	45	77.451	16.244	79.03	41	92
4	60	77.451	21.001	72.88	53	80
5	75	77.451	25.542	67.02	60	73

Table 13: Controlled sensitivity of Task 3 assignment to vendor capacity.

The controlled capacity sweep confirms the expected behavior. Case A welfare remains constant at 77.451 for every capacity value, which confirms that the experiment changes only the limited-capacity constraint.

In Case B, increasing capacity improves the planner outcome. When the capacity per vendor increases from 1 to 5, total vendor capacity increases from 15 to 75, limited-case welfare increases from 5.775 to 25.542, and the number of matched players increases from 15 to 60. At the same time, the number of unmatched players decreases from 118 to 73, and the welfare drop decreases from 92.54% to 67.02%.

Even when the capacity per vendor is 5, Case B welfare remains below Case A. This shows that increasing capacity helps substantially, but limited supply and compatibility constraints still prevent many players from obtaining their best compatible vendors.

5.8 Task 3 conclusion

Task 3 shows how a minimal security set can be used as input to a planner assignment problem. Each secured node becomes a player with a budget, and each vendor has a price, security level, and possibly limited capacity. With infinite vendor supply, most players can choose their best compatible vendor and total welfare is high. With limited capacity, players compete for vendor slots, so the assignment must be solved globally. The implementation handles this case using min-cost flow, where compatible player-vendor edges have cost equal to negative utility and vendor-to-sink edges enforce capacities.

The experiments show that limited capacity causes a large welfare loss and many unmatched players. The unmatched-player diagnosis shows that the main bottleneck is saturation of compatible vendors, not lack of compatibility alone. The sensitivity experiments further show that both the utility trade-off parameter α and vendor capacities affect the final planner outcome. Overall, Task 3 demonstrates that even after a valid security set is selected, resource constraints in the vendor market can strongly affect the achievable welfare.

6 Task 4: Truthful Secure-Path Auction with VCG Payments

6.1 Auction model

Task 4 implements a truthful procurement mechanism for buying a secure path between a source node s and a target node t. The implemented model is a *node-weighted path-procurement problem*: each node is treated as an agent that may participate in the selected path. This modeling choice is consistent with the network-security setting, because nodes represent entities that may incur a cost when they are used in the secure route. An edge-based variant could also be developed, but it is not the model implemented here.

A security set S is used to label nodes as secure or unsecure. Nodes in S are considered secure, while nodes in $V \setminus S$ are considered unsecure. Each node i has a true cost c_i and reports a cost $\hat{c}_i$ to the mechanism.

For an s–t path P, the buyer's score is defined as

$$\text{Score}(P) = \sum_{i \in P'} \hat{c}_i + \lambda |\{i \in P' : i \notin S\}|, \tag{7}$$

where $\lambda \geq 0$ is the penalty for each unsecure node and P' is the set of paid path nodes. In the reported experiments, endpoint exclusion is enabled, so

$$P' = P \setminus \{s, t\}.$$

Thus, the source and target are not paid and do not contribute to the path score.

Equivalently, each paid node has weight

$$w(i) = \hat{c}_i + \lambda \mathbf{1}[i \notin S], \tag{8}$$

while the endpoints have weight zero when endpoint exclusion is enabled. The chosen path is

$$P^\star \in \arg\min_P \sum_{i \in P'} w(i). \tag{9}$$

Therefore, the mechanism does not simply choose the shortest path by number of edges. Instead, it chooses the path with the minimum security-aware reported score, balancing reported node costs and the penalty for using unsecure nodes.

6.2 Implementation procedure

The implementation follows the VCG path-procurement mechanism step by step.

1. The security set S is converted into a secure mask. Nodes in S are labeled secure, and nodes in $V \setminus S$ are labeled unsecure.
2. A true cost c_i is generated for each node. In the truthful baseline, each node reports its true cost, so $\hat{c}_i = c_i$. In the misreport experiment, one selected node is allowed to report a different value.
3. For each node, the mechanism computes the security-aware node weight

   $$w(i) = \hat{c}_i + \lambda \mathbf{1}[i \notin S].$$

   When endpoint exclusion is enabled, the source and target are assigned zero weight:

   $$w(s) = w(t) = 0.$$

4. The node-weighted shortest-path problem is converted into an edge-weighted shortest-path problem. For every undirected edge (u, v), the implementation creates two directed edges. The edge $u \to v$ has weight $w(v)$, and the edge $v \to u$ has weight $w(u)$. Thus, entering a node pays that node's weight.

5. Dijkstra's algorithm is applied to find the path $P^\star$ with minimum total security-aware reported score.
6. The winning agents are the internal nodes of the selected path, $P^\star \setminus \{s, t\}$, because endpoint exclusion is enabled.
7. For each winning node i, the implementation removes i from the graph and recomputes the best s–t path. This gives C_{-i}, the best score achievable without node i.
8. The VCG payment is computed as

$$p_i = C_{-i} - (C^\star - w(i)).$$

9. The profit, or utility, of each winning node is computed as

$$\pi_i = p_i - c_i.$$

This procedure makes the implementation explicit: reports determine node weights, node weights determine the winning path, and VCG payments are computed by comparing the chosen solution with the best alternative obtained when each winner is removed.

6.3 VCG payment rule

After selecting the winning path $P^\star$, the mechanism computes Clarke pivot payments for the selected internal nodes. Let

$$C^\star = \sum_{i \in P^{\star\prime}} w(i)$$

be the optimal score with all nodes available, where

$$P^{\star\prime} = P^\star \setminus \{s, t\}$$

when endpoint exclusion is enabled. For a winning node i, let C_{-i} be the optimal score when node i is removed from the graph. The VCG payment to node i is

$$p_i = C_{-i} - (C^\star - w(i)). \tag{10}$$

This payment represents the externality imposed by node i on the rest of the network. The term $C^\star - w(i)$ is the score of the chosen solution excluding node i's own contribution, while C_{-i} is the best score achievable when node i is unavailable.

The profit of a selected node is

$$\pi_i = p_i - c_i.$$

If removing node i disconnects s from t, then $C_{-i} = +\infty$ and the node is critical. In this implementation, such cases are reported as infinite payments to preserve the Clarke-pivot interpretation. In practical deployments, one would usually avoid or cap such cases by requiring feasible alternative paths or adding additional market rules.

6.4 Example run

As a representative run, the security set produced by Task 1 on a regular graph with $n = 1000$ and $k = 4$ is used. The selected security set contains

$$|S| = 669$$

secure nodes. The auction settings are

$$s = 16, \qquad t = 999, \qquad \lambda = 5, \qquad \text{seed} = 42.$$

Endpoint exclusion is enabled, so the source and target are not paid.

Scenario	Path length	Reported cost	Unsecure nodes	Penalty part	Total score
Truthful baseline	9	274	1	$5 \times 1 = 5$	279
One-liar scenario	10	277	2	$5 \times 2 = 10$	287

Table 14: Task 4 secure-path auction result with $s = 16$, $t = 999$, and $\lambda = 5$.

Under truthful reports, the selected path is

$$[16, 800, 778, 436, 188, 875, 549, 636, 999].$$

Because endpoint exclusion is enabled, the paid internal nodes are

$$[800, 778, 436, 188, 875, 549, 636].$$

The reported costs of these internal nodes sum to 274. Among the paid internal nodes, one node is unsecure. Therefore, the security penalty is

$$\lambda \cdot 1 = 5.$$

The total truthful score is

$$C^\star = 274 + 5 = 279.$$

This example illustrates how the implementation computes the winning path: each candidate path is evaluated by adding the reported costs of its internal nodes and the penalty for unsecure internal nodes. The mechanism selects the path with minimum total score.

6.5 VCG payments in the truthful run

The VCG payments for the truthful winning internal nodes are shown in Table 15. The table reports each winning node's true cost c_i, reported cost $\hat{c}_i$, security-aware weight $w(i)$, alternative cost C_{-i}, VCG payment p_i, and utility $\pi_i = p_i - c_i$.

Node	Secure	c_i	$\hat{c}_i$	$w(i)$	C_{-i}	p_i	π_i
549	Yes	75	75	75	287	83	8
778	No	69	69	74	287	82	13
800	Yes	49	49	49	287	57	8
636	Yes	43	43	43	287	51	8
436	Yes	15	15	15	287	23	8
188	Yes	14	14	14	287	22	8
875	Yes	9	9	9	287	17	8

Table 15: VCG payments and utilities for the truthful winning internal nodes. Here p_i is the VCG payment and $\pi_i = p_i - c_i$ is the node utility.

In this run, removing any one of the truthful winning internal nodes leads to the same best alternative path,

$$[16, 419, 366, 158, 263, 983, 751, 538, 903, 999],$$

with score $C_{-i} = 287$. This is not imposed by the implementation; the alternative path is recomputed separately for each winning node. In this particular instance, the same backup path remains feasible after removing any one of the truthful winning internal nodes.

For example, for node 875 we have

$$w(875) = 9, \qquad C^\star = 279, \qquad C_{-875} = 287.$$

Therefore, its VCG payment is

$$p_{875} = 287 - (279 - 9) = 17.$$

Since its true cost is $c_{875} = 9$, its utility is

$$\pi_{875} = p_{875} - c_{875} = 17 - 9 = 8.$$

This explains where the utility value 8.0000 in the truthfulness check comes from.

6.6 Truthfulness demonstration

The interface also tests one possible misreport. In this run, node 875 is the misreporting agent. Node 875 belongs to the truthful winning path, but after reporting the higher cost 150, it is no longer selected.

Under the modified report vector, the mechanism selects the different path

$$[16, 419, 366, 158, 263, 983, 751, 538, 903, 999].$$

The paid internal nodes on this path are

$$[419, 366, 158, 263, 983, 751, 538, 903].$$

Their reported costs sum to 277. Since two paid internal nodes are unsecure, the penalty is

$$5 \cdot 2 = 10.$$

Thus, the selected path under the misreport has total score

$$277 + 10 = 287.$$

Agent	Utility under truthful report	Utility under lying report
Node 875	8.0000	0.0000

Table 16: Truthfulness check for the tested misreport by node 875.

The tested lie does not improve the agent's utility. Under truthful reporting, node 875 is selected and obtains utility 8.0000. Under the lying report, the selected path changes and node 875 is not selected, so it receives no payment and its utility becomes 0.

This empirical check is only an illustration. It does not prove truthfulness by itself, because it tests only one possible misreport. The truthfulness guarantee comes from the VCG mechanism: the allocation rule minimizes the reported social cost in (7), and payments are computed using the Clarke pivot rule in (10). Under the standard quasilinear utility assumption, truthful reporting is a dominant strategy for each agent in this VCG setting.

6.7 Task 4 conclusion

Task 4 shows how a security set can be used inside a truthful path-procurement mechanism. The selected path balances reported node costs with a penalty for using unsecure nodes. The implementation is node-based: the winning agents are the selected internal path nodes, excluding endpoints when endpoint exclusion is enabled. VCG payments compensate winning nodes according to the externality they impose on the rest of the network.

In the example run, the tested misreport by node 875 does not increase its utility, which is consistent with the theoretical truthfulness of the VCG mechanism. The result should be interpreted as an illustrative check; the truthfulness guarantee follows from the VCG allocation and payment rules rather than from the finite misreport experiment alone.

7 Conclusion

This project studied Network Security Sets through four complementary algorithmic game-theoretic tasks. Task 1 modeled the problem as a strategic game, where Pure Nash Equilibria induce valid inclusion-wise minimal NSS. Experimentally, BRD was the fastest and most robust method, FP often produced smaller final sets but could be iteration-cap sensitive, and RM provided a valid stochastic learning baseline. Task 2 introduced a coalitional perspective: normalized characteristic functions score candidate secured coalitions, Shapley values rank nodes by marginal contribution, and a build–prune procedure converts the ranking into a minimal NSS.

Tasks 3 and 4 used the obtained security sets in downstream mechanisms. Task 3 formulated vendor assignment as a welfare-maximization problem under compatibility and capacity constraints. Task 4 implemented a truthful secure-path procurement mechanism using a node-weighted shortest-path allocation rule and VCG payments.

The implementation supports multiple graph families and is not limited to a single example instance. The main limitations are the cost of Monte Carlo Shapley approximation, the possible iteration sensitivity of stochastic dynamics such as RM and FP, and the node-based formulation of the auction model. Future work could study more efficient learning dynamics, richer characteristic functions, larger capacity-sensitivity experiments, and an edge-based VCG auction variant.